

Capstone Kickoff Document

Human-in-the-Loop Adaptive Alerting for Backend Reliability Monitoring

Working Draft v1 | July 6, 2026

1. Document Purpose

This kickoff document defines the initial system scenario, working research boundary, core metrics, incident taxonomy, and first-phase implementation plan for the capstone project. Its role is to turn the proposal into an actionable starting point for design and development.

2. Working Project Definition

Working topic: A human-in-the-loop adaptive alert noise reduction system for backend reliability monitoring.

Core problem: Backend teams often receive too many low-value alerts, including false positives, repeated alerts, and alerts that are technically valid but operationally unhelpful.

Research objective: Evaluate whether a feedback-driven adaptive alerting approach can reduce alert noise while preserving or improving the detection of meaningful reliability incidents.

3. Monitored System Scenario

The first implementation stage will use a simulated backend reliability environment that mirrors common enterprise execution patterns. The environment will be intentionally small enough to control, but realistic enough to generate meaningful alerts and evaluation data.

3.1 Workload Types

- Scheduled jobs: recurring jobs such as daily aggregation, report generation, cache refresh, and cleanup tasks.
- Batch workflows: multi-step backend pipelines where one failure can affect the full run outcome.
- Queue-based workers: asynchronous consumers that process queued events, retries, and delayed tasks.

3.2 Environment Components

- Job Scheduler module to trigger recurring jobs on a fixed timeline.
- Task Queue module to represent asynchronous backlog, queue delay, and worker saturation.

- Worker Service module to execute jobs and surface retries, timeouts, and partial failures.
- Metrics Generator module to emit Prometheus-style operational metrics.
- Alert Engine module to apply baseline threshold and rule-based alert generation.
- Feedback Module to collect user labels such as useful, noisy, duplicate, expected, false positive, and real incident.
- Adaptive Ranking Module to reprioritize alerts using historical behavior and user feedback.
- Web Dashboard for alert review, explanation display, and administrative configuration.

4. Core Metrics

The first dataset should stay intentionally focused. The following metrics are enough to support alert generation, incident simulation, and comparative evaluation.

- `job_runtime`: execution duration for each job run.
- `job_success_rate`: ratio of successful runs in a defined time window.
- `job_failure_rate`: ratio of failed runs in a defined time window.
- `retry_count`: number of retries required for a job run.
- `timeout_count`: number of timeout events observed for a job or workflow.
- `queue_wait_time`: delay between job submission and execution start.
- `queue_backlog_size`: number of queued tasks waiting for processing.
- `throughput`: completed job count per unit of time.

5. Incident Taxonomy

The project needs a stable incident taxonomy before any alerting logic is implemented. These categories will define how anomalies are injected and how evaluation labels are assigned.

- Latency Spike: execution time rises significantly above the normal range.
- Failure Burst: job failures increase rapidly within a short window.
- Retry Storm: repeated retries occur across multiple tasks or repeated runs.
- Timeout Cluster: multiple timeout events appear for the same workload category.
- Queue Backlog: backlog size and wait time increase together beyond expected operating range.
- Intermittent Failure: unstable failures appear sporadically and create alert noise without clear persistence.

6. Ground Truth and Labels

Because this capstone compares alerting strategies, it must define evaluation labels up front. The simulated environment should therefore produce both metric events and a separate incident log that acts as ground truth.

- **Real Incident:** an injected or observed abnormal event that meaningfully affects latency, success rate, or processing capacity.
- **False Positive:** an alert fired even though the behavior remains within acceptable or expected operating variance.
- **Duplicate Alert:** repeated alerts tied to the same underlying incident within a short time window.
- **Expected Alert:** an alert generated during known maintenance, test injection, or intentionally simulated behavior.
- **Noisy Alert:** an alert that is technically triggered but does not lead to useful operator action.

Practical note: At the prototype stage, these labels can be generated from simulation rules first, then later enriched with manual feedback collected through the UI.

7. Comparative Methods

1. Fixed-threshold alerting: static thresholds trigger alerts without adaptive logic.
2. Rule-based suppression: static grouping, deduplication, and suppression policies reduce repeated alerts.
3. Adaptive feedback-driven alerting: alert priority is adjusted by combining metric behavior, alert history, and user feedback.

Reason for comparison: This three-method structure creates a clear baseline for the capstone and allows the final evaluation to show whether adaptive feedback adds measurable value beyond traditional threshold and rule-based approaches.

8. Phase 1 MVP Boundary

- Three workload types: one scheduled job family, one batch workflow family, and one queue-worker family.
- Eight core metrics only, without logs, traces, or profiling in the first phase.
- Six incident types only, using controlled anomaly injection.
- One basic dashboard containing alert list, alert detail, user feedback, and explanation fields.
- One baseline ruleset and one adaptive ranking workflow.
- Sample data generation and reproducible local execution through Docker Compose.

9. Immediate Next Actions

1. Write the system scenario specification for each workload type, including normal behavior and failure behavior.
2. Define the alert data schema and incident log schema.
3. Create a simulated metric generator that can emit normal runs and injected incidents.
4. Design baseline fixed-threshold and rule-based alert logic.
5. Sketch the minimum UI screens for alert review, feedback entry, and admin configuration.

10. Open Questions

- Should the first prototype center on jobs only, or include queue workers from day one?
- What time windows should define duplicate alerts and suppression rules?
- How much user feedback should be simulated before real interactive labeling is introduced?
- Should the first adaptive method be simple weighted scoring or include lightweight anomaly scoring from the start?